

Developer Opdracht: App-berichten

Na het eerste gesprek willen we je nu ook leren kennen als developer. In plaats van klassieke interviewvragen stellen we je liever een concrete uitdaging, zodat we in ons gesprek kunnen inzoomen op jouw aanpak en keuzes.

De opdracht bestaat uit twee delen: een korte architectuurschets en een kleine Proof of Concept. Voor het technische deel houd je het bij 2 à 3 uur. We kijken niet naar perfectie, maar naar hoe je denkt en structureert.

Context

Onze luisteraars staan via de apps van onze radiostations constant in contact met hun favoriete DJ's. Ze sturen tekstberichten, foto's en audio- of videofragmenten. Voor een DJ is snelheid essentieel: zij moeten in één oogopslag zien wat er binnenkomt, waarbij audioberichten direct worden getranscribeerd zodat ze niet alles hoeven af te luisteren.

Deel 1: Architectuur & Design (High-level)

Stel dat we dit systeem vandaag vanaf nul zouden opbouwen. Beschrijf kort (max. 1 A4 of een aantal diagrammen):

1. **User Interface (App-zijde):** Hoe ziet de flow voor de gebruiker eruit? Focus op een intuïtieve manier om media te versturen.
2. **Backend Architectuur:** Hoe verwerk je de inkomende tekstberichten en eventuele bijgevoegde media? Denk aan opslag, schaalbaarheid en de verwerking van binnenkomende berichten. Beschrijf het proces vanaf een inkomend bericht tot aan het tonen aan de DJ.
3. **DJ & Studio Dashboard:** Hoe presenteert je deze data aan een DJ die live in de uitzending zit? Hoe zorg je dat de transcriptie van audioberichten bijdraagt aan hun workflow? De berichten moeten zo snel mogelijk beschikbaar zijn voor de DJ, wat doe je als het transcriberen (te) lang duurt?

Deel 2: Proof of Concept (Code)

Om de tijd die je hieraan besteedt te beperken (richtlijn: 2 tot 3 uur), vragen we je alleen de core-functionaliteit van de audio-transcriptie uit te werken in een simpele Proof of Concept. Beperk je tot de opdracht hieronder, we verwachten niet de volledige uitwerking zoals geschetst in deel 1.

De opdracht:

Bouw een mini-applicatie (bijv. in Ruby, JavaScript/TypeScript, Rust of Go) die het volgende doet:

- **Endpoint:** Een API-endpoint waar een audiobestand (bijvoorbeeld `.mp3` of `.wav`) naartoe gestuurd kan worden, samen met de naam van de luisteraar.
- **Transcription:** Verwerk de audio via een bestaande API van een transcriptieservice (zoals Whisper of ElevenLabs) tot een transcriptie.
- **Summary:** Maak elk bericht herkenbaar met een korte titel. Deze kan je vormen met een AI-samenvatting van de transcriptie.
- **Frontend:** Een zeer simpele webpagina waar de binnenkomende audioberichten weergegeven worden voor de DJ

Maak bij de implementatie tenminste gebruik van een (bij voorkeur relationele) database voor de persistence van berichten.

Focus op: Functionele code en logische afhandeling. We kijken niet naar zaken als styling of complexe authenticatie. Een werkende "happy flow" is het belangrijkste.

Schrijf de code voor deze opdracht op de manier hoe je dat normaal ook zou doen. Het gebruik van AI voor development mag daar een onderdeel van zijn. Schrijf code waar je trots bent en demonstreer de juiste concepten, patronen en technieken passend bij een dergelijke applicatie.

Het resultaat mag gedeeld worden als link naar een GitHub repo of als zip-bestand.

Transcription services

Als je Whisper of ElevenLabs wil gebruiken heb je een werkende API key nodig. Voor deze opdracht kan je die van ons gebruiken.

- OpenAI - Whisper:
sk-proj-ZZBtYxCa_8MptxJodJuYn7B5RX5zkDcqCe0iauZz-NNGzMaZL4B1bUBNx
pLjtUL1Z5gx6XF6DT3B1bkFJPPYGHSSzEZtemnBPpxXRjWdGMPgrK1rYLHpI2bUa
8Xn085eASYdBrZQGIELnVs3Q2XaC3v15EA
- ElevenLabs: sk_6e3328b171cbd3126598e18c8ade20e53cf6387ab7e24fe2

 **Bijlage:** [Voorbeelden van audioberichten van onze DJs](#)